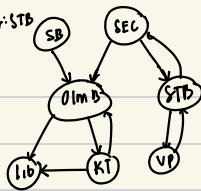


Graduate Enrichment Center: SEC
 Olmsted Building: Olm B
 Science and Tech Building: STB
 Kulkarni Theatre: KT
 Vartan Plaza: VP
 Library: Lib
 Swarnara Building: SB



```

graph LR
    SEC((SEC)) --> OlmB((Olm B))
    SEC --> STB((STB))
    OlmB --> SB((SB))
    OlmB --> KT((KT))
    OlmB --> Lib((Lib))
    STB --> VP((VP))
    KT --> Lib
    SB --> OlmB
    VP --> STB
    Lib --> OlmB
    Lib --> KT
  
```

The function `dfs_all_paths`

at each building start it:

1. Adds it to current path
 2. Marks it as visited (no cycles dont cause problems)
 3. If it goals $\rightarrow$ saves current path
 4. Otherwise $\rightarrow$ explore each neighbour that isn't visited
 5. When done it backtracks (find the goal but in find saves, go to other nodes find other paths) "if start == goal" yield list(path)
- $\rightarrow$ All possible paths from start to goal generated

longest paths = from list of all paths find max (dfs_longest_path)
 shortest paths (dfs_shortest_path) time = from all paths find min

- Because each recursive call represents "stepping into" a new building along the path if goal reached, the current recursive chain holds complete path. When recursive function backtracks automatically. (Also why it tracks nodes so long can be needed)
- Limitations: inefficient. DFS explores all paths, including long ones redundant, DFS finishes long detours even when short route exist

Assumption

- Graph representations: represented in directed graph (adjacency list)
- Each node represents each unique building names
- Equal edge weight: all walkways are equal
- goal must be reachable
- A node can be in different path but not > 1 in one path

interesting findings:

- DFS great for finding all possible paths and longest path since it naturally goes deep and backtracks

DFS, however, is much more efficient for shortest path, since it finds it immediately without generating every possible path.

Cycle detection are critical: without temporarily marking nodes visited, DFS will loop forever

This homework basically a toy campus compared to in real life problems where it would be computationally expensive